

MyTinyC

Краткое введение в язык C

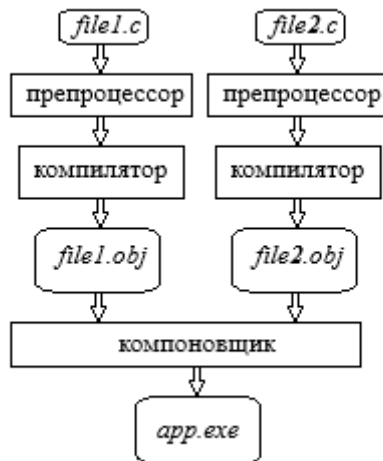
Созыкин А.А. 2025

Оглавление

От .С к .EXE. Этапы создания приложения.....	2
Первая программа.....	3
Упражнение.....	3
Переменные и типы.....	4
Присвоение.....	4
Арифметические операции.....	4
Упражнения.....	5
Сравнение, логика, сдвиг.....	6
Ветвления.....	7
Циклы.....	8
Упражнения.....	8
Указатели.....	10
Массивы.....	10
Строки.....	12
Упражнения.....	13
Функции.....	14
Описание функций.....	14
Использование функций.....	14
Передача параметров.....	14
Упражнения.....	15
Указатели на функции.....	16
Рекурсия.....	16
Как это работает.....	17
Рекурсия и цикл.....	18
Упражнения к разделу "Функции".....	19
Структуры и объединения.....	20
Объединения.....	20
Поля бит.....	20
typedef.....	20
Упражнения.....	20
Директивы препроцессора.....	22
Модификаторы <i>unsigned</i> , <i>short</i> и <i>long</i>	24
Ввод/вывод - клавиатура/консоль (<i>stdio</i>).....	25
Ввод/вывод - файлы (<i>stdio</i>).....	26
Всё на одной странице.....	27

От .C к .EXE. Этапы создания приложения.

C – компилируемый язык. Это значит, что код на языке C (файлы с расширением **.c**, например **MyApp.c**) обрабатываются, каждый отдельно, **препроцессором**, компилируются, также отдельно, **компилятором** в объектный код (файлы с расширением **.obj**), а затем все эти файлы передаются **компоновщику**, который собирает программу в исполняемый файл (с расширением **.exe**). Этот файл и есть готовая программа.



Программы обычно (не всегда) пишут с помощью специальных приложений - «интегрированных сред разработки» IDE (Integrated Development Environment). Такие IDE включают и редактор кода, и компилятор, и компоновщик, да еще отладчик, средства анализа и документирования кода, контроль версий и многое другое. Вот некоторые из популярных IDE:

- Microsoft Visual Studio
- CLion (JetBrains)
- Visual Studio Code (VS Code)
- Qt Creator
- Embarcadero RAD Studio
- Code::Blocks
- Dev C++

Две последние проще других и удобнее для начинающих. Кроме того, есть возможность использовать совсем простенький инструмент - "GDB online". Ничего не надо устанавливать, просто перейти в браузере на <https://www.onlinegdb.com>.

Первая программа

Текст Вашей первой программы на C:

```
1  /******  
2  
3  Welcome to GDB Online.  
4  GDB online is an online compiler and debugger tool for C, C++, Pyth  
5  C#, OCaml, VB, Swift, Pascal, Fortran, Haskell, Objective-C, Assembl  
6  Code, Compile, Run and Debug online from anywhere in world.  
7  
8  *****/  
9  #include <stdio.h>  
10  
11  int main()  
12  {  
13      printf("Hello, world!");  
14      return 0;  
15  }  
16
```

Это и есть первая программа

Щелкаем Run и получаем результат в нижней части окна.

```
input  
Hello World  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Разберем этот код построчно.

Строки 1-8 — комментарий. Все, что между `/*` и `*/`, а также после `//` в строке, компилятор игнорирует. Например: `a = 0; // пожалуй, переменную a самое время обнулить`

9 `#include <stdio.h>`

В C нет встроенных средств ввода-вывода, надо использовать внешние модули. Строка 9 подключает стандартный модуль ввода-вывода. Этот модуль предоставляет нам, в частности, функцию `printf` (вызывается в строке 13).

11 `int main()`

Заголовок главной функции. `"int"` — тип возвращаемого значения (`int` – целое число), скобки — признак того, что это функция, а не переменная.

Программа на C состоит из функций, вне функций операторов нет. Выполнение начинается с функции `main`. Заканчивается выполнение этой функции, - заканчивается выполнение программы. Все остальные функции вызываются внутри `main`.

12 и 15 — фигурные скобки `{}`

То, что внутри фигурных скобок, - блок. В данном случае этот блок — тело функции `main`. Отступы в C только для красоты, их можно использовать как угодно. Или вообще не использовать, а написать программу хоть в одну строчку. Обычно пишут либо так, как в нашем тексте, либо так:

```
int main() {  
    ...  
}
```

13 `printf("Hello, world!");`

Вызываем функцию `printf` из `stdio` и передаем ей единственный аргумент — строковую константу - в двойных (не в одинарных!) кавычках. Одинарные кавычки используются для других целей. Точка с запятой обязательна и **завершает любой оператор** в C. Не разделяет, а именно завершает.

14 `return 0;`

Выход из функции `main`, завершение программы. Возвращаемое функцией `main()` значение — код завершения (`exit code`).

Упражнение

Сотрите текст первой программы и набейте его по памяти.

Переменные и типы

Каждая переменная принадлежит к одному типу, определенному для нее явно. Тип переменной указывается в ее описании и никогда не меняется. Переменная должна быть описана до ее первого использования.

Имена переменных – последовательности английских букв, цифр и знака ‘_’, начинающаяся с буквы или знака ‘_’. ‘A’ и ‘a’ – разные буквы.

Основные типы данных в C: **int** (целые числа), **float** (действительные числа), **char** (один символ). Есть еще: **short**, **long**, **double**, **unsigned int** и др., но это варианты трех основных, оставим их изучение на потом.

Описание начинается с типа и заканчивается списком переменных этого типа (через запятую). В описании можно присваивать переменным начальные значения.

Числовые константы - тип **int** — целые числа и **float** - десятичные дроби (т. н. числа с плавающей точкой). В дробях разделитель целой и дробной части всегда запятая, независимо от настроек операционной системы (напр. 5.3). Возможна «научная» запись числа с плавающей точкой: 3.5e-5 соответствует 3,5 на десять в минус пятой степени.

Константы типа **char** (одиночные символы) выделяются одинарными кавычками. Т.е. ‘q’ и ”q” – совсем не одно и то же.

Примеры описаний:

```
int i, j, k;
int a, b = 3, c = 5, d = -5;
float f = 1.5, g, h, www = 5.e-3, q = .555;
char c = 'a';
```

С помощью функции **sizeof()** можно узнать размер памяти, занимаемой переменной:

sizeof(int) – столько байт занимает любая переменная типа **int**,

sizeof(a) - столько байт занимает переменная **a**.

Переменная существует в пределах того блока, в котором она описана, и во вложенных в него. Если описать переменную вне блоков, она будет видна везде.

```
int global_var; /* видна везде */
{ /* блок 1 */
    int local_var1; /* видна в блоках 1 и 2 */
    { /* блок 2 */
        int local_var2; /* видна только в блоке 2 */
    }
}
```

Если одна переменная описана во внутреннем блоке, а другая - во внешнем, и они имеют одинаковые имена, - видна только описанная во внутреннем блоке.

```
int main(void)
{
    int a = 1;
    {
        int a = 2;
        printf("%d", a); /* будет напечатано 2 */
    }
    printf("%d", a); /* будет напечатано 1 */
}
```

Присвоение

Знак равенства означает присвоение значения выражения справа объекту слева. Слева должен стоять объект, занимающим место в памяти (называется **lvalue** – *locator value*).

Т.е. $c = a + b$ – правильно, а $c + 1 = a + b$ – ошибка.

Выражение вида **a = b** имеет значение, равное **b**. Именно так — выражение имеет значение, не только переменная. Поэтому можно написать

```
int a, b, c;
a = b = c = 1;
c = (a=3) * (b=5); c будет равно 15.
```

Арифметические операции

+ сложение; **-** вычитание; ***** умножение; **/** деление; **%** остаток от деления (деление по модулю).

Запись вида $a=a+b$ можно записать короче: $a+=b$. Это допустимо для всех операций.

Инкремент: $x++$ и $++x$ - увеличение x на 1, но в первом случае увеличение происходит после использования текущего значения в выражении, а во втором — до. Аналогично $--$ - декремент, уменьшение на 1.

Например:

```
int a=2, b=3, c;  
c = a * ++b; /* c будет равно 8 */;  
c = a * b++; /* c будет равно 6 */
```

Переменная b в обоих случаях будет равно 4. Эта операция применима только к объектам *lvalue*, выражение $(a+b)++$; - ошибка.

Нужно помнить, что при операциях, в которых участвуют *int* и *float*, результат преобразуется к *float*, и при присвоении значения целой переменной дробная часть просто **отбрасывается, не округляется** по правилам округления.

Например:

```
int k = 1.9; переменная k получит значение 1  
float k=5/10; переменная k получит значение 0.0 (int/int – int; int пишем во float – отбрасываем дробную часть).  
float k=5./10; переменная k получит значение 0.5 (float/int – float).
```

Тип *char* на самом деле просто целое число длиной 1 байт. Если написать, например, $\text{char } c = 65$;;, ошибки не будет. Это эквивалентно $\text{char } c = 'A'$ (65 – код буквы A). В результате $\text{char } b = 'A' + 1$;;, переменная b будет равна 'B' (код 66).

Упражнения

Модифицируйте текст первой программы, добавив описания переменных a , b , c и f . Для вывода используйте оператор `printf("a=%d b=%d c=%c f=%f", a, b, c, f);`

```
#include <stdio.h>  
int main()  
{  
    int a = 1, b = 2;  
    char c = 'A';  
    float f = 0.5;  
  
    a = 1; b = 2; c = 'A'; f = 0.5;  
    a += b*b + a*a; b = 10*f/5*a; c += a; f = a/(b+1)*10.;  
    printf("a=%d b=%d c=%c f=%f\n", a, b, c, f);  
  
    a = 1; b = 2; c = 'A'; f = 0.5;  
    a = 'C' - c; b *= 43%(a+b); c += f * 3; f *= 1/a;  
    printf("a=%d b=%d c=%c f=%f\n", a, b, c, f);  
  
    a = 1; b = 2; c = 'A'; f = 0.5;  
    a = b++ * ++b; c += '0'; f *= (a * b * f);  
    printf("a=%d b=%d c=%c f=%f\n", a, b, c, f);  
  
    return 0;  
}
```

Попытайтесь предсказать, что выведет программа. Затем выполните программу и сравните Ваше предсказание с результатом. Если не совпадут, обязательно разберитесь!

Сравнение, логика, сдвиг

Булевский (логический) тип переменных и соответствующие константы в языке C отсутствуют. «Ложью» считается ноль. Любое другое значение - «истина».

Сравнение:

“==” - равно,
“!=” - не равно,
“>” - больше,
“<” - меньше,
“>=” - не меньше,
“<=” - не больше.

Примеры:

```
int c = (1 == 2); /* c равно 0 */
int c = (3 == 3); /* c не равно 0 */
int c = (1 < 2); /* c не равно 0 */
```

Логические операции:

&& - “И” - конъюнкция,
|| - “ИЛИ” - дизъюнкция (две вертикальные палочки, не I (ай) и не l (эль),
!= - “НЕ” - отрицание.

Примеры:

```
a > 0 && a < 1 /* a в интервале (0,1) */
a >= 'A' && a <= 'Z' /* a - заглавная английская буква */
a != 0 || b != 0 /* хотя бы одна из переменных a и b отлична от 0 */
```

Есть еще побитовые операции: | - побитовое “ИЛИ” и & - побитовое “И”. Они применяются к каждой паре соответствующих битов в представлении операндов в памяти.

Примеры:

`int a = 1 || 2;` `a` не будет равна 0, потому что 1 - истина (не 0), 2 - истина (тоже не 0), истина ИЛИ истина — истина.

`int b = 1 | 2;` `b` будет равна 3, потому что 1 — это несколько нулей и единица - ...001 (в памяти числа хранятся в двоичной системе счисления), 2 — несколько нулей и 10 — ...010, 0 ИЛИ 0 есть 0, 1 ИЛИ 0 – 1, 0 ИЛИ 1 – тоже 1, итого 001 | 010 дает 011 в двоичной системе, т.е. 3 в десятичной.

$$\begin{array}{r} \dots 0001 \\ | \\ \dots 0010 \\ \hline \dots 0011 \end{array}$$

`int c = 1 && 2;` `c` - истина И истина - истина,

`int d = 1 & 2;` `d` будет равно 0 (001&010, а 1 И 0 есть 0),

$$\begin{array}{r} \dots 0001 \\ \& \\ \dots 0010 \\ \hline \dots 0000 \end{array}$$

`int a = 5; int c = a & 1;` `c` получит значение последнего бит двоичной записи числа `a`, для 5 это 1; можно использовать для проверки числа на четность */

В C есть операции, позволяющие сдвинуть содержимое ячейки памяти влево или вправо на несколько бит: <<n и >>n, где n - целое.

Примеры:

```
int a = 1, c; c = a << 2; 0001 → 0100 /* то же, что c = a*4; но во много раз быстрее */
int a = 1 << n; /* 2 в степени n */
```

Ветвления

Условный оператор

```
if(условие1) /* скобки обязательны */
{ блок1 }
else if(условие2)
{ блок2 }
else
{ блок3 }
```

Условия — выражения (не забываем, 0 - ложь, все остальное - истина).

Если условие1 не 0, выполняется блок1;

если условие1 равно 0, а условие2 не 0, выполняется блок2;

если условие1 и условие2 равны 0, выполняется блок3.

Части *else* могут отсутствовать.

Примеры:

```
if( a > b ) c = 1; (блок может состоять из одного оператора, тогда {} можно не ставить)
if( a > b && c )
{
    a = b;
    c = 0;
}
else
    b = 0;
```

Тернарный оператор:

(условие ? выражение1 : выражение2).

Если условие истинно, результат равен выражение1, иначе выражение2.

Например:

```
int x = 5, y = 7;
int z = ( x > 0 ? 1 : -1 ) * ( y + x < 10 ? 0 : x ); /* результат: z = 1 * x = 5 */
```

Переключатель:

```
switch(целое выражение)
{
    case целая константа 1:
        блок 1
        break;
    case целая константа 2:
        блок 2
        break;
    .....
    default:
        блок 3
}
```

Выполняется

блок 1, если выражение равно константе 1;

блок 2, если выражение равно константе 2;

блок 3, если ни одна из констант не равна выражению.

Количество секций *case* не ограничено. Если в одной из секций опустить *break*, будут выполнены все следующие блоки до первого *break* или до конца.

Циклы

for(оператор1; условие; оператор2) { тело цикла }

Условие — произвольное выражение (помним: 0 — ложь, все остальное — истина).

Алгоритм работы.

- 0 Выполняется оператор1.
- 1 Вычисляется условие. Если не 0, выполняется тело цикла, иначе — выход из цикла.
- 2 Выполняется оператор2.
- 3 Переход к п.2.

Т.е. оператор1 - подготовка к циклу, инициализация; оператор2 - переход к следующей итерации; условие - критерий окончания.

Пример. Найти сумму целых чисел от 0 до 10 включительно.

```
int i, s = 0;
for( i = 0; i < 11; i++ ) s += i;
```

Любой из операторов и условие можно опустить, сохранив ";", но без условия цикл будет бесконечным.

while(условие) { тело цикла } Тело цикла выполняется, пока условие не нуль. Эквивалентно **for(;условие;) { тело цикла }**

do { тело цикла } while(условие); То же, что *while*, но тело цикла выполняется один раз до первой проверки условия.

continue - переход к следующей итерации, т.е. к проверке условия;

break - немедленный выход из цикла.

Упражнения

1. Что выведут эти программы? Предскажите, набейте и убедитесь.

Факториал

```
int i, x = 5, y = 1;
for( i = 1; i <= x; i++ ) y *= i;
printf("%d", y);
```

Сумма геометрической прогрессии

```
float x = 5, q = 0.5, s; /* x - первый член, q - знаменатель */
for( s = 0; x > 1.e-9; x*=q ) s += x; // суммируем, пока следующее слагаемое > 10-9
printf("%f", s);
```

2. Напишите программу, которая распечатывает внутреннее представление целого числа. Для ввода целого числа *x* используйте оператор `scanf("%d", &x);` для печати - `printf("%d", x)`.

Это сложное задание, поэтому решение ниже. Постарайтесь сначала сами. Если покажется слишком сложно - можно пропустить. Обязательно разберите решение! Поиграйте с этой программой. Обратите внимание на представление отрицательных чисел.

```

1  #include <stdio.h>
2
3  /* идея: с помощью операции & числа с маской, содержащей только одну 1,
4     выделяем соответствующий этой единице бит.
5     Например: x & ...0010 дает значение второго справа бита числа x,
6     остальные биты обнуляются операцией И с нулями маски */
7
8  int main()
9  {
10     int x, n, m; /* описание переменных:
11                  x - введенное число,
12                  m - битовая маска - в одной позиции 1, в остальных - 0,
13                  n - позиция 1 в маске */
14
15     /* приглашение и ввод числа x */
16     printf("Целое число: ");
17     scanf("%d", &x);
18
19     /* собственно решение */
20     n = sizeof(x)*8-1; /* сначала в маске 1 в крайней левой позиции */
21     for(m = 1<<n; m >= 0; m = 1<<--n) printf("%d", x&m? 1:0);
22
23     return 0;
24 }

```

```
Целое число: 5
00000000000000000000000000000000101
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

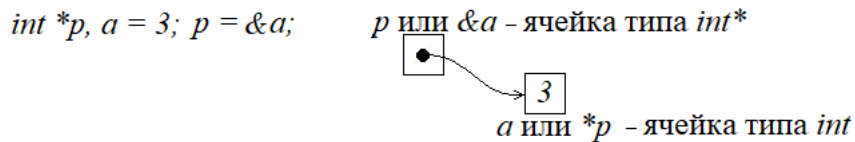
Указатели

Указатель — это переменная, содержащая адрес в памяти. При работе с указателями используются символы `*` и `&`:

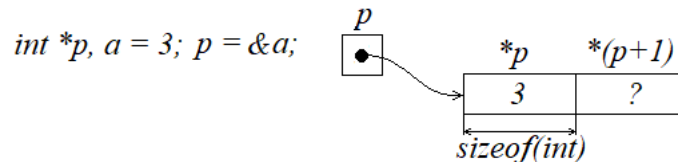
`&a` — указатель на переменную `a` (адрес переменной `a`),

`*p` — содержимое ячейки, адрес которой записан в переменной `p` - "значение по указателю `p`".

Описание переменной, предназначенной для хранения указателя, содержит `*` перед именем переменной. Например, `int *p;` - описание переменной `p`, в которой будет храниться указатель на переменную типа `int`. Эта запись как бы говорит: «То, на что указывает значение переменной `p` (т.е. `*p`), имеет тип `int`». В этом случае переменная `p` **имеет тип `int*`**. На рисунке в ячейке `a` (тип `int`) содержится число 3, а `p` (тип `int*`) хранит адрес этой ячейки ("указывает на неё").



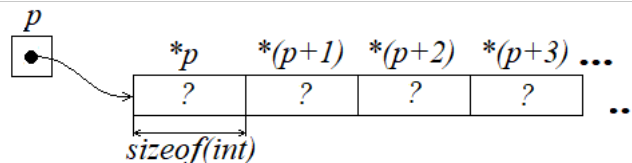
К указателям применимы операции сложения и вычитания (`+`, `-`, `++`, `--`, `+=`, `-=`). При увеличении указателя на 1 адрес увеличивается на длину переменной соответствующего типа, т. е. `p+1` — указатель на следующую за `*p` ячейку памяти того же типа.



Массивы

Массив - непрерывная цепочка ячеек одного типа в памяти и указатель на первую ячейку из этой цепочки. Память для этой цепочки нужно получить от операционной системы (ОС). Это можно сделать, например, с помощью функции `malloc` из `stdlib`.

```
#include <stdio.h>
#include <stdlib.h> /* подключаем модуль с malloc() */
int main()
{
    int n;
    scanf("%d", &n); /* запрашиваем у пользователя кол-во элементов будущего массива */
    int *p = malloc(n*sizeof(int)); /* запрашиваем у ОС память для массива - n ячеек типа int.
                                   Теперь p - указатель на первую из них. */
    ..... /* используем массив */
    free(p); /* возвращаем память */
}
```

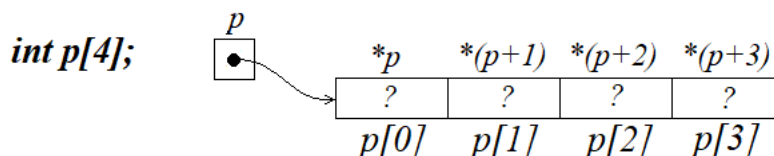


Такие массивы называются "динамическими".

Если Вы уже при написании программы знаете, сколько элементов массива Вам понадобится, можно описать этот массив как "статический":

`int p[константное выражение];`

Например:



В этом случае память будет выделена заранее, при старте (загрузке) программы.

i-ый элемент **любого** массива p : $*(p+i)$ или $p[i]$ (это одно и то же).

Не забудьте, что номер последнего элемента на 1 меньше количества элементов в массиве, а номер первого элемента - 0! Выход за пределы - ошибка при выполнении программы.

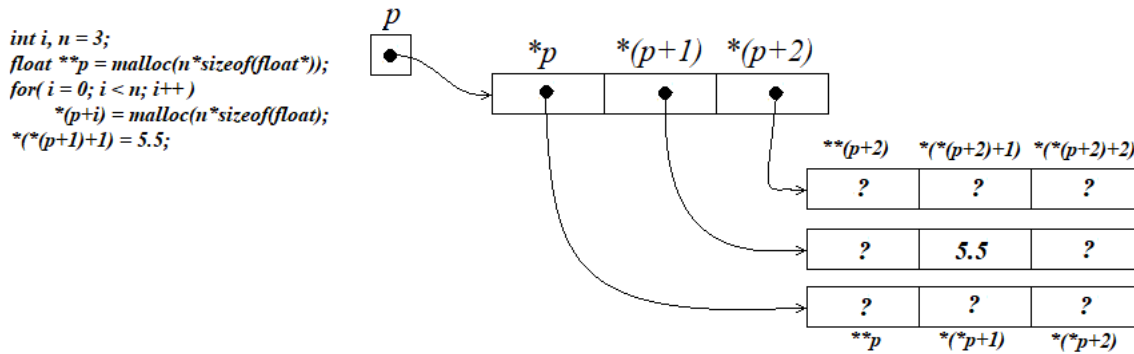
Пример. Всем элементам массива p присвоить 0.

```
int i, p[n];  
for( i = 0; i < n; i++ ) p[i] = 0;
```

Статический массив можно инициализировать: $\text{float } p[] = \{ 1.0, 2.5, 3.0 \};$ /* массив из 3-х элементов */

Двумерный **статический** массив описывается так: $\text{float } p[3][3];$ элемент такого массива: $p[i][j]$.

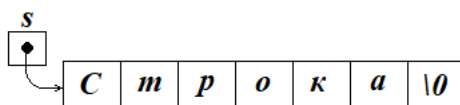
Вариант реализации двумерного **динамического** массива:



Строки

Строка в C - массив типа *char* с нулевым байтом '\0' на конце.

char *s = "Строка";



Для работы со строками подключите `<string.h>`.

Символ '\0' - нулевой байт - 8 нулей - один из т.н. "управляющих символов" (сравните: 0 - это *int* - 4 байта, '0' - цифра, код 48).

Полезно помнить еще два управляющих символа: '\n' - перевод строки и '\t' - табуляция.

Не забудьте, что строка — это указатель, поэтому результат сравнения, например, "a" и "b" может оказаться неожиданным:

```
main.c
1 #include <stdio.h>
2 int main()
3 {
4     if("b" > "a") printf("b > a");
5     else printf("a > b");
6     return 0;
7 }
8
input
a > b
...Program finished with exit code 0
Press ENTER to exit console.
```

Пример 1

Сосчитать слова в строке. Для ввода строки используем

fgets. У этой функции три параметра:

- указатель_на_начало_области_в_памяти,
- размер_этой_области,
- имя_потока_ввода).

Печатаем с помощью ***puts***(строка).

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int i, n, /* искомое кол-во слов */
7      len; /* длина строки */
8
9      char *s = malloc(1024); /* память для строки.
10                             Некоторые компиляторы могут потребовать
11                             приведения к типу, тогда
12                             char *s = (char*)malloc(1024); */
13      puts("Input string: "); /* приглашение и ввод строки */
14      fgets(s, 1024, stdin); /* s может содержать символ перевода строки в конце */
15
16      for( len = 0; s[len] != '\0' && s[len] != '\n'; len++ ); /* определяем длину строки */
17
18      for( i = 1, n = 0; i <= len; i++ )
19      { /* считаем концы слов - предыдущий символ не пробел, а текущий - пробел или конец */
20          if( s[i-1] != ' ' &&
21              (s[i] == ' ' || s[i] == '\n' || s[i] == '\0') )
22              n++;
23      }
24
25      printf("n = %d", n);
26      return 0;
27 }
28

```

Вместо `char *s = malloc(1024)` можно, конечно написать `char s[1024]`.

Пример 2

Перевести строку в верхний регистр (только английский алфавит).

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  int main()
4  {
5      int i;
6      char *s = malloc(1024);
7      puts("String: ");
8      fgets(s, 1024, stdin);
9
10     for( i = 0; s[i]; i++ )
11         if( s[i] >= 'a' && s[i] <= 'z' ) s[i] += 'A' - 'a';
12     puts(s);
13     return 0;
14 }
15

```

Здесь используется то, что английские буквы от А до Z закодированы подряд:

'B' == 'A'+1, 'C' == 'A'+2 ..., и 'b' == 'a'+1, 'c' == 'a'+2..., но не обязательно строчные следуют за заглавными. Следовательно, например, 'Q' = 'q' + ('A'-'a').

Упражнения

Упражнение 1.

- 1) Ввести три целых числа — *nx*, *ny*, *nz*.
- 2) Создать **динамический** трехмерный массив *nx* x *ny* x *nz*.
- 3) Для всех $0 \leq i \leq nx$, $0 \leq j \leq ny$ и $0 \leq k \leq nz$ элементу $[i][j][k]$ этого массива присвоить значение $i+j+k$.
- 4) Ввести *i*, *j*, *k* и вывести значение элемента массива $[i][j][k]$.
- 5) Освободить память, выделенную под массив.

Упражнение 2. Ввести строку и распечатать все слова в столбик, переведя каждую первую букву слова в верхний регистр, а все остальные буквы слова — в нижний.

Функции

Сразу после загрузки программы в память операционная система (ОС) ищет функцию **main()**, где бы она ни находилась, и начинает ее выполнять. Дойдя до конца этой функции или встретив оператор **return**, программа завершается.

Описание функций

Описание функции состоит из заголовка и тела функции - блока. Структура заголовка:

тип_возвращаемого_значения имя_функции(список_формальных_параметров).

Параметры в списке параметров - тип и имя, через запятую. Пример заголовка:

int MyFunc(int a, int b, float c).

Заголовок должен встретиться раньше, чем функция будет вызвана первый раз. Обычно заголовки (**только** заголовки, их называют "прототипами") всех функций помещают в отдельный файл, т.н. "заголовочный", - "header file" с расширением ".h". Кроме прототипов, в этот файл помещают и другие данные, описания структур, например. В код программы этот файл подключается с помощью директивы препроцессора "#include" (см. ниже). Препроцессор просто вставляет указанный файл на место этой директивы.

Файл MyProg.h

```
#include <stdio.h>

/* Заголовки функций - "прототипы" */
int main(void);
int MyFunc( int a, int b, float c );
float FtoDo( int *p, float a);
```

Файл MyProg.c

```
#include "MyProg.h"

int main(void) { блок - тело функции main, может содержать вызовы MyFunc и F1q }

int MyFunc( int a, int b, float c ) { блок - тело функции MyFunc }
float FtoDo( int *p, float a ) { блок - тело функции FtoDo }
```

Функцию нельзя описывать внутри другой функции.

Список параметров может быть пустым или **void**, что тоже означает отсутствие параметров.

Использование функций

Функции могут использоваться в выражениях:

int a = 5 * MyFunc(3, 2*2, x+y) + MyFunc(1, 2, q); или как операторы: **MyFunc(3, 2*2, 0.5);**

В последнем случае возвращаемое значение теряется. Если функция не должна возвращать значение, ее описывают как **void**.

void Doesnt_Return_Anythink(cнусок_параметров)

Такая функция не может использоваться в выражении - только как оператор.

Оператор **return** прекращает выполнение функции и возвращает управление вызывающей программе. Если функция должна возвращать значение, оно указывается за этим оператором.

return возвращаемое_значение; или return(возвращаемое_значение);

и такая строка должна встретиться в теле функции хотя бы один раз. Причем возвращаемое значение должно быть того же типа, которое указано в описании, или приводиться к нему, иначе - ошибка.

В функциях типа **void** - просто **return;** или его может вообще не быть. В последнем случае функция завершается после выполнения последнего оператора.

Передача параметров

Параметры, перечисленные в **заголовке**, называются "формальными". При вызове функции на их место подставляются значения - "фактические" параметры. Иногда формальные параметры называют просто параметрами, а фактические - аргументами. Внутри тела функции формальные параметры используются как локальные переменные. При вызове функции на место каждого формального параметра ставится соответствующее значение - выражение для фактического. **Значение переменной, передаваемой функции в качестве параметра, в вызывающей функции измениться не может.**

Пусть есть функция:

```
int Func(int a) { a++; return a; }
```

Вызывающая функция:

```
int a = 3;
int b = Func(a);
printf( " a = %d  b = %d", a, b);
```

Будет напечатано: " a = 3 b = 4 ". Т.е. *a* в вызывающей функции и *a* внутри *Func* - разные переменные.

Примеры параметров разных типов.

```
int Func(
    int n,           /* целое число */
    int *q,          /* указатель на целое или массив целых */
    int **w,         /* двумерный массив целых, массив указателей */
    char *t          /* строка (массив символов) */
);
```

Вызывающая функция:

```
int n = 3, q[5], w[15][15], t = "Строка";
int k = Func(n, q, w, t); или так: Func(1, q, w, "String");
```

Так это что же, функция не может вернуть больше одного значения?! Да, это так! Но...

Если передать в функцию указатель, он не изменится (фактический параметр), но переменной, на которую он указывает, измениться ничто не мешает. Так что ограничений тут нет!

Функция: `int Func(int *a) { (*a)++; return *a; }`

Вызывающая функция:

```
int a = 3;
int b = Func(&a);
printf( " a = %d  b = %d", a, b);
```

Будет напечатано: " **a = 4** b = 4 ".
a

Упражнения

1. Напишите функцию решения квадратного уравнения с таким прототипом:

```
int QuadraticEquation( float a, float b, float c, float *x1, float *x2);
```

Здесь *a*, *b*, *c* - коэффициенты уравнения, *x1* и *x2* - корни или нули, если корней нет.

Возвращает 1, если корни есть, иначе - 0.

Справка: квадратный корень вычисляется функцией

```
#include <math.h>
```

```
float sqrt(float);
```

2. Напишите функцию, переводящую дату в формате ГГГГ-ММ-ДД в ДД.ММ.ГГГГ. Имейте в виду, что день и/или месяц во входной дате могут задаваться одной цифрой, т.е. может встретиться и ГГГГ-М-ДД, и ГГГГ-ММ-Д.

3. Напишите функцию, вычисляющую количество гласных и согласных букв во введенной строке (только английский алфавит).

Указатели на функции

Функция - код в памяти, и у этого кода, конечно, есть адрес. Есть, следовательно, **указатель на функцию**, который можно использовать как любой другой указатель. Например, создавать массивы функций или передавать функцию в другую функцию как аргумент.

Рассмотрим небольшой искусственный пример.

```
1  #include <stdio.h>
2
3  // функция F1 - возвращает удвоенное значение аргумента типа int
4  int F1(int n) { return(n*2); }
5
6  // первый аргумент - указатель на функцию int, принимающую int; второй - int
7  // печатает результат функции f с аргументом a
8  void F2(int (*)(int n), int a) { printf("F2: %d", f(a)); }
9
10 void main(void)
11 {
12     int a, b;
13     int (*p)(int n); // объявление указателя на функцию int, принимающую int
14     p = F1; // указателю присваивается значение - F1
15     a = (*p)(5); // вызов функции по указателю
16     b = p(5); // другой вызов функции по указателю
17     printf("%d %d\n", a, b);
18     F2(p, 5); // передача указателя на функцию в качестве аргумента
19 }
20
```

будет напечатано:

10 10
F2: 10

В строке 13 описан указатель на функцию. Имя **указателя** - *p*. В строке 14 этому указателю присваивается значение. Общий вид описания указателя на функцию:

тип_возвращаемого_значения (*имя_указателя)(список аргументов_функции)

Сравните: *int *p(float a)* - это прототип функции, возвращающей указатель на целое, а не описание указателя на функцию.

Строки 15 и 16 - примеры вызова функции по указателю. В строке 8 описана функция, получающая указатель на другую функцию в качестве первого аргумента, а в строке 18 - вызов этой функции.

Рекурсия

Рекурсию обычно объясняют на примере вычисления факториала. Последуем этой традиции.

По определению факториалом целого положительного числа называется произведение всех целых чисел от 1 до этого числа включительно:

$$n! = \begin{cases} 1, & \text{если } n=0 \\ n*(n-1), & \text{если } n>0 \end{cases}$$

Например, $5! = 5*4! = 5*4*3! = 5*4*3*2! = 5*4*3*2*1! = 5*4*3*2*1*0! = 5*4*3*2*1*1 = 120$.

А что, если так и написать на C?

```
1 #include <stdio.h>
2
3 int Fact(int n)
4 {
5     if( n == 0 ) return 1;
6     else return n*Fact(n-1);
7 }
8
9 int main()
10 {
11     int n = 5;
12     printf("%d", Fact(n));
13     return 0;
14 }
15
```

$n! = \begin{cases} \text{если } n = 0, & 1; \\ \text{иначе } n*(n-1)! \end{cases}$

input
120
...Program finished with exit code 0
Press ENTER to exit console.

Получилось! Функция может вызывать сама себя. **Это и есть рекурсия.**

Алгоритм применения рекурсии можно представить так.

1. Вызов рекурсивной функции с параметрами, соответствующими первой итерации.
2. Проверка условия продолжения:
 - если нужно продолжать
 - подготовить следующую итерацию;
 - вызвать функцию, т.е. повторить вычисления с новыми параметрами;
 - иначе
 - вернуться в вызывающую функцию.

Как это работает

Чтобы рекурсия, которая на самом деле призвана обеспечить простоту и ясность кода, не казалась чем-то особенным и непонятным, рассмотрим подробнее механизм вызова функции.

Транслятор и компоновщик переводят код на С в последовательность команд процессора, - исполняемый файл. В заголовке этого файла содержится также информация о памяти, которую должна выделить ОС этой программе при ее загрузке. Часть этой памяти - "стек" (stack), остальное - "куча" (heap). Стек организован по принципу "FILO" - first in last out (первым вошел - последним вышел) - как стопка тарелок или тупик на железной дороге. Размер стека можно указать в настройках IDE, но обычно оставляют значение по умолчанию (1Мб).

Итак, какие операции закладываются при трансляции нашего кода для вызова функции $B()$ из функции $A()$? **Процессор же ничего о функциях языка С не "знает"**. Как, впрочем и о существовании этого языка.

1. Текущее состояние функции $A()$ - значения локальных переменных и текущая команда - помещаются в стек - на "вершину".
2. Вычисляются значения параметров функции $B()$.
3. Управление передается команде, соответствующей началу функции $B()$. Если $B()$ - это та же $A()$, то на начало кода $A()$. Т.е. **в этом случае выполняется тот же код, но с другими значениями переменных. Как если бы $B()$ была просто точной копией $A()$.**

Когда $B()$ завершается, из стека извлекается состояние функции $A()$, и вычисления в пределах $A()$ продолжаютс с того места, где они были прерваны и с теми же значениями локальных переменных.

Так что, если Вам трудно осмыслить рекурсию, представьте, что рекурсивный вызов функции - это вызов другой функции, которая является точной копией вызывающей. Только и всего.

Между прочим, что произойдет, если условие продолжения будет выполняться слишком долго или даже всегда? Да, - "**stack overflow**" - переполнение стека.

Рекурсия и цикл

Но если рекурсия - это просто повторение каких-то действий с разными параметрами... а цикл тогда что? Разве не то же самое?

```
for(  
    подготовка первой итерации;  
    условие продолжения;  
    подготовка следующей итерации  
)  
    повторить вычисления с новыми параметрами ("тело цикла")
```

Да, так и есть, **рекурсию всегда (? , попробуйте доказать) можно заменить циклом.**

Какой вариант быстрее: цикл или рекурсия? Цикл быстрее, - манипуляции со стеком требуют времени. Так зачем рекурсия? Для красоты и простоты кода. Вот как с факториалом в начале этого урока - если алгоритм задан рекурсией, а это бывает очень часто, проще так его и кодировать, если время не очень критично.

Упражнения к разделу "Функции"

Задача 1

"Целое_без_знака" определяется как "<цифра> или <целое_без_знака><цифра>".
Написать функцию, которая получает строку и возвращает 1, если эта строка - целое_без_знака, или 0 в противном случае.

Задача 2

Если Вы не знакомы с этой нотацией, можно пропустить.

Функция $\sin(x)$ определяется как бесконечная сумма

$$\sin(x) = \sum_{n=1}^{\infty} (-1)^{n-1} \frac{x^{2n-1}}{(2n-1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

или

$$\sin(x) = \sum_{n=1}^{\infty} q_n(x) \quad \text{где} \quad q_1 = x; \quad q_n = -\frac{x^2}{2(n-1)(2n-1)} q_{n-1}$$

Написать два варианта функции вычисления $\sin(x)$ - с рекурсией и без нее. Суммирование должно прекращаться, когда модуль q_n будет меньше одной тысячной от текущей суммы: $|q_n| / \sin(x) < 0,001$.

Задача 3

Тоже можно пропустить.

Написать функцию, которая принимает три параметра: x и указатели на функции $\sin(x)$, написанные для задачи 2, а возвращает отношение времени выполнения варианта с рекурсией ко времени выполнения варианта без рекурсии. Для работы со временем можно использовать функцию, определенную **time(NULL)** в **<time.h>**, которая возвращает количество секунд с 00:00 01.01.1970.

Структуры и объединения

В С структура рассматривается как новый тип данных.

Определение структуры:

struct имя_структуры { тип_поля имя_поля; ... };

Точка с запятой в конце обязательна.

Например: ***struct STR { int a; float b; };***

По сути это и есть определение нового типа данных. Можно объявлять переменные этого типа и указатели на переменные этого типа. При этом имя типа будет:

"struct имя_структуры".

Например: ***struct STR s, *ps;*** - переменная *s* типа *struct STR* и указатель *ps* на *struct STR*.

Не все операции применимы к переменным этого нового типа: *s1 = s + *ps;* - ошибка.

Существуют также константы этого нового типа - список значений полей через запятую в фигурных скобках.

Например: ***struct STR s = {5, 3.5};***

Определение структуры и описание переменных можно совместить:

struct STR { int a; float b; } s, *ps;

Разумеется, полями структур могут быть другие структуры, кроме нее самой:

struct STR { int a; struct STR s; }; - **ошибка**, но полем структуры может быть указатель на такую же структуру:

struct STR { int a; struct STR *s; }; - допустимо.

Элементы структуры можно использовать как переменные: если ***struct STR { int a; ... } s, *ps;***, то *s.a* - значение поля *a* переменной *s*, а *ps->a* - значение поля *a* переменной, на которую указывает указатель *ps*.
Т.е. *ps->a* - синоним *(*ps).a*.

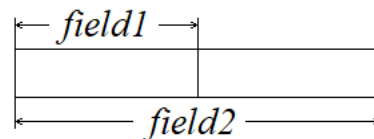
Объединения

Если в структуре поля хранятся друг за другом, то в объединении - друг на друге. Т.е. все поля начинаются с одного адреса.

struct S { int field1; float field2; };



union U { int field1; float field2; };



Поля бит

В структурах и объединениях можно использовать поля нестандартного размера - т.н. "битовые поля".

тип имя_поля : длина_в_битах;

Например: ***struct S { int two_bits: 2; int six_bits: 6; };***

typedef

Это ключевое слово дает возможность назначать синонимы типам данных

typedef описание_типа_данных имя_типа_данных;

Например:

```
typedef struct
{
    char *first_name;
    char *last_name;
    int age;
} PERSON;
```

Теперь *PERSON* - новый тип данных, и можно объявить переменную типа *PERSON*:

PERSON p; или массив *PERSON ps[1024];* и обращаться к полям этой структуры: *p.age = 35;*

Упражнения

1. Напишите структуру для объекта "человек" - *PERSON*, включающую имя, отчество, фамилию, год рождения, пол, родителей. Родители тоже люди (*PERSON*).
2. Если есть переменная типа *PERSON* из предыдущего задания, как получить имя деда того человека, которого эта переменная описывает?
3. Создать новый тип переменной: указатель на функцию, принимающую два аргумента типа *float* и возвращающую целое число.

Директивы препроцессора

Компиляции исходного файла предшествует его обработка **препроцессором**. Препроцессор преобразует исходный файл с С-кодом в другой файл с С-кодом, который и передается компилятору. Специальные директивы в исходном коде определяют, в чем именно состоит это преобразование. Эти директивы начинаются со знака **#** в **первой позиции** строки и **НЕ** заканчиваются точкой с запятой.

Здесь будут рассмотрены только несколько основных директив.

#include <файл> или **#include "файл"**

Препроцессор просто вставляет указанный файл вместо этой директивы. В угловых скобках указываются стандартные файлы вроде *stdio.h*, в кавычках - все остальные. Файлы в кавычках препроцессор ищет сначала в текущем каталоге, а затем в системных каталогах ОС, но можно указать полный путь к файлу.

#if константное_выражение 1

строки 1

#elif константное_выражение 2

строки 2

#else

строки 3

#endif

Если константное выражение 1 - не нуль, строки 1 остаются в коде, строки 2 и 3 удаляются; иначе если константное выражение 2 - не нуль, строки 2 остаются в коде, строки 1 и 3 удаляются; иначе строки 3 остаются в коде, строки 1 и 2 удаляются.

Константное выражение - это выражение, значение которого известно до компиляции (препроцессор-то работает до компилятора).

#define и #undef

Определение макроса и отмена определения. Макросом называется идентификатор, который следует за **#define**.

Например: **#define DEBUG** - определение макроса *DEBUG*.

Эта директива имеет три формы.

Форма 1

#define имя_ макроса

Макрос с именем "имя_макроса" считается определенным до появления директивы

#undef имя_ макроса

Форма 2

#define имя_макроса последовательность_символов

Препроцессор подставляет "последовательность_символов" вместо "имя_макроса" во всем файле.

Например: **#define false 0** означает, что все вхождения слова **false** будут заменены на **0**. Макрос с именем имя_макроса считается определенным.

Форма 3

#define имя_макроса(список_параметров) последовательность_символов

Список_аргументов - перечень идентификаторов через запятую. Препроцессор ищет в коде фрагмент имя_макроса(список_значений_параметров), подставляет вместо параметров в "последовательность_символов" их значения, а затем заменяет этой последовательностью найденный фрагмент.

Например: **#define MIN(x,y) (x<y? x:y)**

в коде фрагмент **int mn = MIN(a+b, a*b);** будет заменен на **int mn = (a+b<a*b?a+b:a*b);**

#ifdef макрос

строки 1

#else

строки 2

#endif

Если макрос определен, строки 1 **остаются в коде**, строки 2 **удаляются**; иначе строки 2 **остаются в коде**, строки 1 **удаляются**.

Есть вариант

#ifndef макрос

строки 1

```
#else
строки 2
#endif
```

Если макрос НЕопределен, строки 1 **остаются**, строки 2 **удаляются**;
иначе строки 2 **остаются**, строки 1 **удаляются**.

Примеры использования

Пример 1

Проект состоит из четырех файлов, *file1* включает *file2* и *file3*, и все три включают *file4*.

```
file1.c
#include "file1.h"
#include "file2.h"
#include "file3.h"
#include "file4.h"
```

```
file2.c
#include "file2.h"
#include "file4.h"
```

```
file3.c
#include "file3.h"
#include "file4.h"
```

Получается, что *file4* должен быть вставлен в *file1* три раза, что почти наверняка означает ошибку. Решение:

```
file4.h
#ifndef FILE4
#define FILE4
все остальное содержимое файла
#endif
```

Когда файл будет вставляться первый раз, макрос *FILE4* еще не будет определен, т.е. будет выполнено условие для *#ifndef*, а значит останется директива *#define FILE4* и все содержимое файла. При каждой следующей попытке вставить этот файл макрос *FILE4* будет определен, и все содержимое файла в код не попадет.

Пример 2

В процессе отладки, особенно больших программ, программист вставляет в код множество строк, содержащих вывод промежуточных данных, сообщений типа: "Вход в функцию *Qqq*" и т.п. Когда отладка закончена, эти строки надо находить и убирать, что может оказаться трудоёмкой задачей. Решение - при вставке в код отладочных строк обрамлять их директивами *#ifdef* - *#endif*:

```
... код программы
#ifdef DEBUG
отладочные действия
#endif
... код программы
```

Тогда при отладке достаточно поместить в самое начало программы директиву
#define DEBUG
а по окончании отладки убрать ее, и препроцессор удалит всю отладочную информацию.

Упражнения

- 1) Напишите макросы, позволяющие использовать в коде переменные типа *bool* с константами *true* и *false*. Чтобы можно было написать, например, такой код: ***bool a = true, b = false;***
- 2) Напишите макрос *ABS* - модуль числа. Чтобы код ***int a = ABS(-5); printf("%d", a);*** выводил 5.

Модификаторы *unsigned*, *short* и *long*

Целые числа (*int* или *char*) хранятся в памяти в т.н. дополнительном коде, в котором левый бит кодирует знак: 0 - положительное число, 1 - отрицательное. В один байт (*char*) можно записать числа от -128 (-2^7) до 127 (2^7-1), в 4 байта (*int*) - от -2147483648 (-2^{31}) до 2147483648 ($2^{31}-1$). Чтобы левый символ не трактовался как знак, перед типом надо поставить модификатор ***unsigned***. Тогда в байт можно записать числа от 0 до 255.

Например: ***unsigned int a; unsigned char c;***

Тип можно опустить, тогда подразумевается *int*: ***unsigned a;*** - то же, что ***unsigned int a;***

Модификаторы ***short*** и ***long*** определяют размер ячейки для целого числа:

long - длинное, обычно слово (64 для 64-разрядной системы), ***short*** - короткое. Впрочем разные компиляторы могут трактовать эти модификаторы по-разному. Посмотрите, как обстоят дела с этим у Вас. Например для gdb online вот так:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("short - %lu байт\n", sizeof(short));
6     printf("int - %lu байт\n", sizeof(int));
7     printf("long - %lu байт\n", sizeof(long));
8
9     return 0;
10 }
```

input

```
short - 2 байт
int - 4 байт
long - 8 байт
```

Ввод/вывод - клавиатура/консоль (stdio).

Печать: `int printf( строка_форматирования, значение, ... );` Возвращает количество напечатанных символов. Печатается строка форматирования, в которую вместо полей, начинающихся с %, подставляются значения: первое поле слева - первое значение, второе поле - второе значение и т.д. Каждое поле содержит символ для типа соответствующего значения:

`%d - int, %u - unsigned, %c - char, %s - char*, %f - float...`

Символ \ используется для отмены специального смысла следующего за ним символа: \" - кавычки, \' - апостроф, \\ - сам \... Чтобы напечатать % напишите %%.
Специальные символы: \n - новая строка, \r - в начало строки, \t - табуляция.

Для типа float можно указать общее количество символов и количество цифр после запятой.

Пример

```
main.c
1 #include <stdio.h>
2 int main()
3 {
4     int i = 5;
5     char *s = "Hello, \"USER\"";
6     float f = 3.1415927;
7     char *wd = "C:\\tmp\\print\\";
8     printf(" int: %d \n string: %s \n float %9.2f \n path: %s",
9           i, s, f, wd);
10 }
```

Печать кавычек
*Печать *
Перевод строки
Длина поля в символах - 9, цифр после запятой - 2

```
input
int: 5
string: Hello, "USER"
float      3.14
path: C:\tmp\print\
```

Ввод: `scanf( строка_форматирования, указатель, ... );`

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 int main()
4 {
5     int i;
6     char *s = malloc(1024);
7     float f;
8     scanf("%d %s %f", &i, s, &f);
9     printf(" int: %d \n string: %s \n float %f", i, s, f);
10 }
```

```
input
5
Hello
3.1415927
int: 5
string: Hello
float 3.141593
```

scanf
printf

Все как при выводе, но в строке форматирования только поля с типами, а вместо значений - указатели.

Ввод/вывод - файлы (stdio).

Сначала пример:

```
main.c  TEMP.DAT  ⋮
1  #include <stdio.h>
2  #include <stdlib.h> /* для malloc */
3
4  int main()
5  {
6      /* запись */
7      FILE *f1 = fopen("TEMP.DAT", "w"); /* открываю TEMP.DAT для записи */
8      fprintf(f1, "Hello"); /* пишу в файл */
9      fclose(f1); /* закрываю файл */
10
11     /* чтение */
12     f1 = fopen("TEMP.DAT", "r"); /* открываю TEMP.DAT для чтения */
13     char *s = malloc(1024); /* память для строки из файла */
14     fscanf(f1, "%s", s); /* читаю строку */
15     fclose(f1); /* закрываю файл */
16     printf("%s", s);
17
18     return 0;
19 }
```

input

Hello

...Program finished with exit code 0
Press ENTER to exit console.

Прежде чем работать с файлом - читать или писать - его надо открыть, а после работы - закрыть. **Открыть** можно функцией

FILE *fopen(путь_к_файлу, режим_доступа);

Варианты "режим_доступа":

- "r" - чтение из **существующего** файла;
- "r+" - чтение и запись, файл должен существовать;
- "w" - запись, если файл не существует - создается, если существует - стирается;
- "w+" - запись и чтение, если файл не существует - создается, если существует - стирается;
- "a" - дописывание, если файл не существует - создается;
- "a+" - чтение и дописывание, если не существует - создается.

Возвращает указатель на объект **FILE**, который потом используется для работы с открытым файлом. **Закреть** файл: **int fclose(file);** где *file* - указатель, который вернула функция **fopen**.

Запись и чтение - **fprintf** и **fscanf** - такие же, как **printf** и **scanf**, но первый параметр - указатель, полученный при открытии файла (см. пример).

Библиотека **stdio** содержит большое количество функций управления вводом/выводом, но, во-первых, полное описание этой библиотеки легко найти в сети и, во-вторых, в C++ все иначе, хотя эти возможности сохранены, так что если изучение C не более чем ступень к C++, на этой библиотеке не стоит слишком задерживаться.

```
int i;
FILE *infile, *outfile;
infile = fopen("file1.dat", "r");
outfile = fopen("file2.dat", "w");
while( fscanf(infile, "%d", &i) != EOF )
    fprintf(outfile, "%d", i);
fclose(outfile);
fclose(infile);
```